

Expressions in Our Programming Language

(Data types)

- **Primitive data types** are: int (integers) and bool (Boolean).
- **Composite types:** (multi-dimensional) Arrays of primitive types of values, with integer indices.
 - Our purpose is to learn about program verification, so we keep the programming language as simple as possible.

(Expressions)

- An **expression** is a piece of code who can be evaluated to some value. In our programming language, expressions have primitive type values. For example, you can consider some non-quantified predicate as an expression with Boolean value. Expressions in our programming language can be built from:
 - **Expressions**
 - **Constants:** Integers (0, 1, -1 ...) and Boolean constants (T and F).
 - **Variables of primitive types**
 - **Functions** that return **primitive type values**
 - **Operations:**
 - On integers: +, -, *, / ($\div$), %, $\text{sqrt}()$, ^ ... and =, $\neq$, <, $\leq$, >, $\geq$, ...
 - ❖ Note that, / and $\text{sqrt}()$ round towards 0 to an integer. For example, $13/3 = 4$, $13/(-3) = -4$, and $\text{sqrt}(17) = 4$.
 - ❖ Division and mod by 0 and sqrt of negative values generate runtime errors.
 - On Booleans: $\neg$, $\wedge$, $\vee$, $\rightarrow$, $\leftrightarrow$, ...
 - On arrays: $\text{size}()$ and array element selection. For example, $b = (0,3,4)$, then $\text{size}(b) = 3$.
 - **Arrays:**
 - As usual, $b[e]$ is array element selection. Note that, e is an expression that evaluates to an integer.
 - $\text{size}(b)$ gives the length of b .
 - In a multi-dimension array, $b[e_0][e_1] \dots [e_{n-1}]$ is selecting the element with index e_0 in the first dimension, e_1 in the second dimension ... e_{n-1} in the n^{th} dimension. Note that, n is not a variable but an integer constant here. ("..." is not understandable). For example, $b = ((6,3), (2,5,8))$, then $b[1 * 1][1 + 1] = 8$.
 - Note that we can never have an array appear by itself in an expression, it is always wrapped in some function, or we are selecting some element in the array. (Why?)
 - **Conditional:** **if B then e_1 else e_2 fi**
 - Semantically, if B evaluates to true, then evaluate e_1 ; if B evaluates to false, then evaluate e_2 .
 - Note that, e_1 and e_2 are expressions and we require them to have the same type.
 - There is also "**if - else if - else**" in our programming language, it is written as "**if B_1 then e_1 else B_2 then e_2 else e_3 fi**".
- Note that:
 - We don't explicitly declare variables; we assume that we can infer the types. For example: to have expression $p \vee x > 0$, we don't need something like "create variable x of type *int*" we can simply use an assignment statement such as " $x := 5$ " to start using x .

- An expression must evaluate to a primitive type of value, so it cannot evaluate to an array. For example: (assuming a and b are two arrays) **if B then a else b fi**[0] is illegal. But **if B then $a[0]$ else $b[0]$ fi** is legal.
- Functions who return primitive type of values are allowed in an expression, but an expression cannot yield a function. For example: **if B then $f(x)$ else $g(x)$ fi** is legal; but **if B then f else g fi** (x) is not.

1. Are the following expressions legal?

- | | |
|---|-----|
| a. $x \% b[y]$ | Yes |
| b. $a[0 : 2]$ | No |
| c. if $x < 0$ then $x * x$ else $\text{sqrt}(x)$ fi $+ y$ | Yes |
| d. if $x < 0$ then p else T fi $+ y$ | No |
| e. if $i < 0$ then $b[0]$ else $i \geq \text{size}(b)$ then $b[\text{size}(b) - 1]$ else $b[i]$ fi | Yes |

(Notations)

- Most commonly, c and d are constants; e and s are general expressions; B and C are Boolean expressions; a and b are array names; and u, v , etc. are variables. Greek letters like α and β stand for semantic values.

(Evaluate an expression)

- In general, evaluation is a process to translate texts (“something syntactic” or something a computer doesn’t understand immediately) to values (“something semantic” or something a computer can understand immediately).
- With a proper state, an expression can be evaluated to a value of primitive type.
 - For example: $\sigma = \{x = 5, y = 2\}$, then $\sigma(x * y) = 10$. Here, $x * y$ is an expression that we want to evaluate, and σ is a state that’s proper for $x * y$.
- The value of $\sigma(e)$ depends on what kind of expression e is, so we use recursion on the structure of e (the base cases are variables and constants, and we recursively evaluate sub-expressions).
 - $\sigma(x)$ = the value that σ binds variable x to. For example, if $\sigma = \{x = 5\}$, then $\sigma(x) = 5$
 - $\sigma(c)$ = the value of the constant c . For example, $\sigma(5) = 5$. (σ is irrelevant here.)
 - $\sigma(e_1 + e_2)$ = the value $\sigma(e_1)$ plus the value $\sigma(e_2)$ [and similar for $-$, $*$, $/$ etc.].
 - $\sigma(e_1 < e_2) = T$ **iff** the value $\sigma(e_1)$ is less than the value $\sigma(e_2)$ [similar for $\leq$, $=$, etc.].
 - $\sigma(e_1 \wedge e_2) = T$ **iff** the value $\sigma(e_1)$ and the value $\sigma(e_2)$ are both T [similar for $\vee$, $\rightarrow$ etc.].
 - $\sigma(\text{if } B \text{ then } e_1 \text{ else } e_2 \text{ fi}) = \sigma(e_1)$ if the value $\sigma(B)$ is T ; it = $\sigma(e_2)$ if the value $\sigma(B)$ is F .
- As an aside, here we have a question, when we evaluate an expression, how does something “syntactic” become something “semantic”? In other words, how is a piece of code compiled into something meaningful to us? In this small section, to make this clear, I will use highlights to show the values (which are something semantic).

2. Let $z \equiv 2 + 3$, evaluate z in an arbitrary state σ .

$$\sigma(z) \equiv \sigma(2 + 3) = \sigma(2) + \sigma(3) = 2 + 3 = 2 + 3 = 5$$

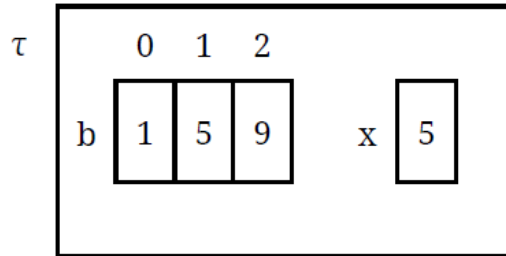
3. Let $\sigma = \{x = 1\}$, let $\tau = \sigma \cup \{y = 1\}$, and let $e \equiv (x = \text{if } y > 0 \text{ then } 17 \text{ else } y \text{ fi})$, evaluate e in state τ .

$$\begin{aligned} \tau(e) &\equiv \tau(x = \text{if } y > 0 \text{ then } 17 \text{ else } y \text{ fi}) \\ &= (\tau(x) = \tau(\text{if } y > 0 \text{ then } 17 \text{ else } y \text{ fi})) \end{aligned}$$

$$\begin{aligned}
&= (\mathbf{1} = \tau(\text{if } \tau(y) > \tau(0) \text{ then } 17 \text{ else } y \text{ fi})) \\
&= (\mathbf{1} = \tau(\text{if } \mathbf{1} > \mathbf{0} \text{ then } 17 \text{ else } y \text{ fi})) \\
&= (\mathbf{1} = \tau(\text{if } \mathbf{1} > \mathbf{0} \text{ then } 17 \text{ else } y \text{ fi})) \\
&= (\mathbf{1} = \tau(\text{if } \mathbf{T} \text{ then } 17 \text{ else } y \text{ fi})) \\
&= (\mathbf{1} = \tau(17)) \\
&= (\mathbf{1} = \mathbf{17}) \\
&= (\mathbf{1} = \mathbf{17}) \\
&= \mathbf{F}
\end{aligned}$$

(Arrays and their values)

4. How to express the following state τ ?



○ $\tau = \{b[0] = 1, b[1] = 5, b[2] = 9, x = 5\}$

We can take the value of an array to be a function from index values to stored values.

○ $\tau = \{b = \beta, x = 5\}$ where $\beta(0) = 1, \beta(1) = 5, \beta(2) = 9$

If we give the function a name β , then we can write τ like this. A finite function is considered semantic.

○ $\tau = \{b = \beta, x = 5\}$ where $\beta = \{(0, 1), (1, 5), (2, 9)\}$

The function β can also be expressed as a collection of tuples (index, stored value), which is also semantic.

○ $\tau = \{b = \beta, x = 5\}$ where $\beta = (1, 5, 9)$

The function β can also be simplified to a sequence of values.

○ $\tau = \{b = (1, 5, 9), x = 5\}$

5. Let $\sigma = \{x = 1, b = \beta\}$ where $\beta = (2, 0, 4)$, evaluate $b[x + 1] - 2$ in σ .

$$\begin{aligned}
\sigma(b[x + 1] - 2) &= \sigma(b[x + 1]) - \sigma(2) \\
&= \sigma(b)(\sigma(x + 1)) - \mathbf{2} \\
&= \sigma(b)(\sigma(x) + \sigma(1)) - \mathbf{2} \\
&= \sigma(b)(\mathbf{1} + \mathbf{1}) - \mathbf{2} \\
&= \sigma(b)(\mathbf{1} + \mathbf{1}) - \mathbf{2} \\
&= \mathbf{\beta(2)} - \mathbf{2} \\
&= \mathbf{4} - \mathbf{2} \\
&= \mathbf{4} - \mathbf{2} \\
&= \mathbf{2}
\end{aligned}$$

Updating a State

- For any state σ , variable x , and value α , the “update” of σ at x with α , written $\sigma[x \mapsto \alpha]$, is the state that is a copy of σ except that it binds variable x to value α .
 - Note that, we are not really updating σ itself (although that is the traditional way to call this operation), that’s why we quote the word “update”: $\sigma[x \mapsto \alpha]$ is a new state and σ is not changed.
 - We can give $\sigma[x \mapsto \alpha]$ a new name but we don’t have to. We read $\sigma[x \mapsto \alpha](v)$ left-to-right — we’re looking at the state $\sigma[x \mapsto \alpha]$ and applying it to variable v .
6. Let $\sigma = \{x = 1, y = 2\}$, answer the following questions about state τ .
- a. Let $\tau = \sigma[x \mapsto 3]$, then $\tau = \{x = 3, y = 2\}$.
 - b. Let $\tau = \sigma[z \mapsto 3]$, then $\tau = \{x = 1, y = 2, z = 3\}$.
 - $\sigma(z)$ doesn’t need to be defined (z is bind with a variable in σ) before updating σ .
 - c. Let $\tau = \sigma[x \mapsto 1]$, then $\tau = \{x = 1, y = 2\}$.
 - τ and σ are consist of the same bindings, they are not syntactically equivalent though.
7. True or False
- a. If $\sigma(x)$ is not defined, then $\sigma[x \mapsto 0] = \sigma \cup \{x = 0\}$.
True
 - b. If $\sigma(x)$ is defined and $\sigma(x) \neq 0$, then $\sigma[x \mapsto 0] = \sigma \cup \{x = 0\}$.
False, $\sigma \cup \{x = 0\}$ becomes ill-formed since x appears twice.
 - c. Let $\sigma = \{x = 5\}$, then $\sigma[x \mapsto 0] \models x \geq x^2$.
True
 - d. Let variables $x \neq y$ be both bound in σ , then $\sigma[x \mapsto 0](y) = \sigma(y)$
True.
 - e. Let $\sigma = \{x = 5\}$, then $\sigma[x \mapsto 2 + 1] = \{x = 3\}$
True, $2 + 1$ is a semantic value. Remember that a function who returns a primitive type is also semantic.
 - f. Let $\sigma = \{x = 5\}$, then $\sigma[x \mapsto x + 1] = \{x = 6\}$
False, we cannot bind a variable with an expression (something syntactic), it becomes ill-formed.
 - g. Let $\sigma = \{x = 5\}$, $\sigma[x \mapsto \sigma(x + 1)] = \{x = 6\}$
True.